

Static class constructor

1. Content

- [1. Content](#)
- [2. Abstract](#)
- [3. Motivation](#)
- [4. Impact On the Standard](#)
- [5. Design Decisions](#)
 - [5.1. Declaration and definition](#)
 - [5.2. Parameters](#)
 - [5.3. This pointer](#)
 - [5.4. Execution order](#)
 - [5.5. Inheritance](#)
 - [5.6. Initializer list](#)
 - [5.7. One Definition Rule](#)
 - [5.8. Auto generation](#)
 - [5.9. Local classes](#)
 - [5.10. Static destructor](#)
- [6. Static inline variables](#)
- [7. References](#)

2. Abstract

As a class constructor is dedicated for object initialization, proposed static class constructor offers type registration. When class defines static class constructor, its code is executed once, early before *main()* function execution. As usual static member function, static class constructor has no associated object and has access to all the static members of the class.

3. Motivation

Static class constructor allows execution of class code, without calling it from outside the class body. This feature provides a flexible and object oriented way to register new functionality of an application. Currently it can be achieved through creation of a dummy global variable, where default class constructor of that variable performs additional code for such registration. Nevertheless, without static class constructor, this cannot be achieved in header-only code. Please refer to [Design Decisions](#) for example usage. The static class constructors offer:

- consistent and object oriented mechanism for type registration
- static member initialization through initializer list of static class constructor
- execution of class code without calling it directly from outside the class body

4. Impact On the Standard

This proposal adds new language feature which affects a compiler only.

The feature is backward compatible as classes do not have to provide a definition of static class constructor. When class defines a static class constructor, used syntax is identical with declaration and definition of default class constructor, except that the static class constructor name is preceded by *static* keyword.

Static class constructor is executed once for each class defining such constructor. It happens exactly in the same manner as for constructors of objects defined at global scope. Both are always executed before execution of *main()* function. Program startup code uses the same mechanism to execute both, constructors of global variables, and static class constructors. For static class constructors pointer to associated object can be undefined or set to *NULL*, as *this* keyword is not available for static class constructors.

Static class variables can be initialized on member initializer list of static class constructor. Therefore, when single header is used for multiple translation units, One Definition Rule must be met.

5. Design Decisions

For better clarification, please consider the following code using a static class constructor:

```
class MyPlugin : public Plugin {
public:
    static MyPlugin() {
        static MyPlugin myPlugin;
        Plugins::registerPlugin(&myPlugin);
    }
    ~MyPlugin() {
        Plugins::unregisterPlugin(this);
    }
};
```

Which is equivalent to the following, when defined at global scope:

```
class MyPlugin : public Plugin {
public:
    MyPlugin() {
        Plugins::registerPlugin(this);
    }
    ~MyPlugin() {
        Plugins::unregisterPlugin(this);
    }
} myPlugin;
```

The equivalent code have some pitfalls which can be avoided with static class constructor:

- There is no need to create dummy global variable to perform initialization.
- Each dummy global variable used in initialization have associated identifier and therefore is visible under IntelliSense. It becomes more difficult in navigation for projects using multiple dummy global variables.
- Static constructors allows to perform early initialization for headers-only libraries, where global variables cannot easily be used.

5.1. Declaration and definition

Class does not have to provide a declaration nor definition of static class constructor. When class defines a static class constructor, used syntax is identical with declaration and definition of default class constructor, except that the static class constructor name is preceded by static keyword.

5.2. Parameters

The static class constructor is called once by program startup code. It cannot be executed manually and its address cannot be obtained. It does not take any arguments as in case of default class constructor.

5.3. This pointer

The static class constructor is not associated with an object as other static class functions. Therefore *this* pointer is not available for static class constructors.

5.4. Execution order

Static class constructors are executed one by one among execution of class constructors for variables defined at global scope in order of consecutive definitions occurring in translation unit.

5.5. Inheritance

Static class constructors are inherited exactly in the same manner as in case of default class constructors. Consecutive actions are taken in following order:

- execution of static class constructors of base classes in order from the top most class,
- initialization of static members not defined through the initializer list,
- initialization of static members defined through the initializer list,
- execution of static class constructor code.

5.6. Initializer list

The static class constructor is a good place to provide initialization of static class members. It can be achieved through initializer list as for non-static class member initialization on non-static class constructor initializer list.

5.7. One Definition Rule

When single header is used for multiple translation units, One Definition Rule must be met to avoid static member redefinition.

5.8. Auto generation

The static class constructor can initialize static class members through initializer list as non-static members are initialized through class constructor. However in case when default class constructor is not defined for class, object members are initialized with its default constructors. This cannot be achieved for static class constructor, to be backward compatible. Therefore, if static class constructor is not defined, static class member variable must be defined outside the class body as usual. Otherwise, when static class constructor is defined, all static members not defined through initializer list will be initialized with its default constructors, even if they are not defined outside the class body.

5.9. Local classes

There is no distinction between static class constructors defined for classes at global and local scope.

5.10. Static class destructor

A static class destructor was also considered on early proposal stage. It could however work in similar way to static class constructor, but to perform type deregistration. It has been found that introducing such functionality can be simply replaced with classical destructor, as static class constructor allow to create a static object, which will be destroyed on process termination.

Nevertheless it would be very useful to have a static class destructor along with static class constructor, as it may open new possibilities which were not so obviously visible at the time when this document was created.

6. Static inline variables

The static class constructor offers similar features to static inline variables proposed for C++17 standard. However both proposals are not excluding each other. As static inline variables are dedicated for static class member initialization, achieving early initialization functionality through static inline variables is more difficult and have more complicated syntax. In this case static class constructor is more reasonable.

7. References

- ISO C++ Standard (ISO/IEC 14882) 2003, section 3.2.
- Hal Finkel and Richard Smith, Inline Variables, 20150407, N4424, <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2015/n4424.pdf>